

APBD 2026 | Tutorial 7

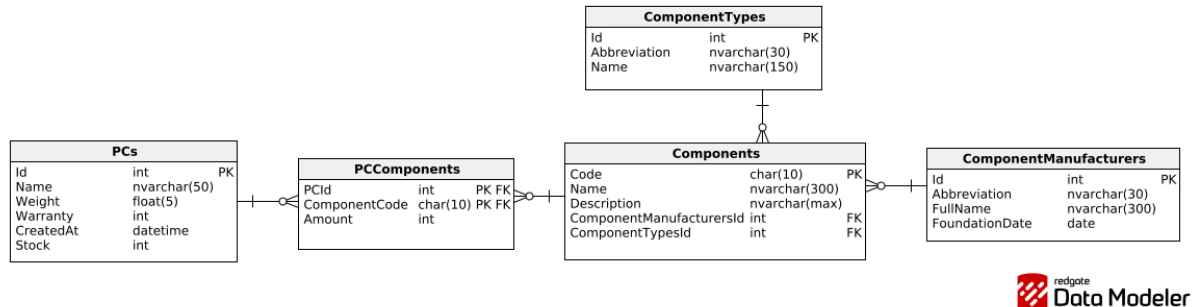
Below is the content of the exercise.

REST API for managing computers and components using EF Core Code First

The goal of the task is to create a **REST Web API** application used to manage computers and their components. The project will be implemented using the ASP.NET Core platform and **Entity Framework Core** in the **Code First approach**. Based on the entity classes you define, a database structure is automatically created, along with relationships between the tables.

Code First

Based on the database diagram shown, you need to prepare an application model in the **Entity Framework Core Code First approach**. To do this, you need to create the appropriate entity classes that map the tables from the diagram and configure the relationships between them, such as one-to-many and many-to-many relationships.



Additionally, you need to implement the required restrictions and configurations, including:

- primary keys,
- foreign keys,
- required fields,
- maximum lengths of text data,
- Data types
- compound keys,
- and the corresponding navigation properties.

Once the models are ready, you need to create a database context (**AppDbContext**) and generate a database structure using **Entity Framework Core migrations**.

In addition, the database should be filled with sample seed data. You should prepare a **minimum of 3 records** for each table, maintaining the correct relationships between the entities. Data

seeding should be accomplished using the HasData() mechanism or another solution available in Entity Framework Core.

Endpoint scope

- GET api/pcs
- GET api/pcs/{id}/components
- POST api/pcs
- PUT api/pcs/{id}
- DELETE api/pcs/{id}

Endpoint logic

GET api/pcs

Endpoint returns a list of all computers along with basic information such as name, weight, warranty period, date created, and stock status.

Example answer `GetAllResponse.json`:

```
[
{
  "id": 1,
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
},
{
  "id": 2,
  "Name": "Office Mini Pro",
  "weight": 4.2,
  "warranty": 24,
  "createdAt": "2026-04-15T13:30:00",
  "stock": 12
}
]
```

GET api/pcs/{id}/components

Endpoint returns a list of components assigned to the selected computer with the specified ID. The result should contain information about the components and their quantity in a given computer set. If the computer does not exist, return a 404 Not Found.

An example answer is provided in the `GetByIdResponse.json` file.

POST api/pcs

Endpoint should allow you to add a new computer. The computer data should be passed in the body of the request, and after saving it correctly, the record should be added to the database.

Example content of the body request `PostRequest.json`:

```
{
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
}
```

Example `PostResponse.json` answer:

```
{
  "id": 1,
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
}
```

PUT api/pcs/{id}

Endpoint allows you to edit the data of an existing computer based on the provided ID. The modified information provided in the request should be stored in the database.

Example content of the `body PutRequest.json` request:

```
{
  "name": "Gaming Beast X",
  "weight": 12.5,
  "warranty": 36,
  "createdAt": "2026-05-08T09:00:00",
  "stock": 5
}
```

DELETE api/pcs/{id}

The endpoint should allow you to remove the computer with the indicated ID from the database along with the bindings if the relationships have been configured properly in Entity Framework Core. If the computer does not exist, return **404 Not Found**. Once removed correctly, return **204 No Content**.

Remember:

During the implementation of the project, special attention should be paid to good programming practices and the correct organization of the application.

- Use the appropriate layering architecture, remembering to separate the application into layers:
 - business logic,
 - and the API/presentation layer.
- Don't directly share database entities in API endpoints. Prepare separate **DTO (Data Transfer Object) classes** for:
 - data incoming to the application,
 - data returned by the API,
 - and data creation and update operations.
- Configure and leverage Entity Framework Core **migrations** to create and update the database structure. All generated migrations should be added to the project repository.
- API endpoints should return appropriate HTTP statuses, consistent with the result of the operation performed, including:
 - **200 OK** - correct download or modification of data,
 - **201 Created** - correct creation of the resource,
 - **204 No Content** - correct deletion of data,
 - **400 Bad Request** - incorrect input,
 - **404 Not Found** - no indicated resource,
 - **500 Internal Server Error** - server/application error.
- Ensure that the relationships are properly configured and that the data is in the database.
- The project code should be readable, properly divided into directories, and compliant with the naming conventions of C# and ASP.NET Core.
- The project must use asynchronous operations (**async/await**) when communicating with the database.